



K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering & Technology



OPERATING SYSTEM

LABASSINGMENT 2 - REPORT

Program: BCA(AI & DS)

Course Code – ENCA252 (2026-2027)

Submitted By

Name - Aryan Maurya

Rollno - 2401201028

Course - BCA (AI & DS)

Submitted To

Jyoti Yadav

1. Introduction

Deadlock is a critical problem in operating systems where a group of processes become permanently blocked because each process is waiting for a resource held by another. This situation leads to system deadlock where no process can proceed.

The Banker's Algorithm, proposed by Edsger Dijkstra, is a deadlock avoidance technique that works like a bank managing loans. It ensures the system remains in a safe state by simulating resource allocation before actually granting requests.

This report presents the implementation and analysis of the Banker's Algorithm covering:

- System Input and Data Representation
- Need Matrix Calculation
- Safety Algorithm Implementation
- Safe Sequence Determination
- Result Analysis

2. Learning Objectives

- Understand deadlock concepts in operating systems
- Implement Banker's Algorithm for deadlock avoidance
- Calculate the Need Matrix using Maximum - Allocation
- Determine safe/unsafe state using the Safety Algorithm
- Find the safe sequence of process execution

3. Tools & Technology

- Programming Language: Python 3.x
- Libraries Used: os, time (built-in)
- IDE: Visual Studio Code
- Platform: Linux (Ubuntu) via VirtualBox

Task 1 — System Input and Data Representation

The program accepts system data including the number of processes, number of resource types, Allocation Matrix, Maximum Matrix, and Available Resources from the user.

Input Data Used:

Allocation Matrix:

Process	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Maximum Matrix:

Process	R0	R1	R2
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

Available Resources: [3, 3, 2]

Task 2 — Need Matrix Calculation

The Need Matrix is calculated using the formula:

$$\text{Need}[i][j] = \text{Maximum}[i][j] - \text{Allocation}[i][j]$$

Need

	Need R0	Need R1	Need R2
P0 P1 P2 P3	7	4	3
P4	1	2	2
	6	0	0
	0	1	1
	4	3	1

The Need Matrix shows how many more resources each process requires to complete its execution.

Task 3 — Banker's Safety Algorithm Implementation

The Safety Algorithm determines whether the system is in a safe state by checking if a safe sequence exists.

Algorithm Steps:

- Step 1: Initialize Work=Available = [3, 3, 2]
- Step 2: Initialize Finish[i]= False for all processes
- Step 3: Find process i where Finish[i] = False and Need[i] <= Work
- Step 4: Work = Work+Allocation[i], set Finish[i] = True
- Step 5: Repeat until all processes finish or no progress
-

Safety Algorithm Execution:

Process	Need	Work Before	Work After
P1	[1, 2, 2]	[3, 3, 2]	[5, 3, 2]
P3	[0, 1, 1]	[5, 3, 2]	[7, 4, 3]
P4	[4, 3, 1]	[7, 4, 3]	[7, 4, 5]
P0	[7, 4, 3]	[7, 4, 5]	[7, 5, 5]
P2	[6, 0, 0]	[7, 5, 5]	[10, 5, 7]

Task 4 — Safe Sequence Determination

After the Safety Algorithm completes successfully, the safe sequence of execution is determined.

Safe Sequence: $P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$

Explanation of Safe Sequence:

- P1 executes first as its Need $[1,2,2] \leq$ Available $[3,3,2]$
- P3 executes next as its Need $[0,1,1] \leq$ Work $[5,3,2]$ P4
- executes as its Need $[4,3,1] \leq$ Work $[7,4,3]$ P0 executes
- as its Need $[7,4,3] \leq$ Work $[7,4,5]$ P2 executes last as its
- Need $[6,0,0] \leq$ Work $[7,5,5]$

Task 5 — Result Analysis

System State:

System is in a SAFE STATE

Analysis:

The Banker's Algorithm successfully found a safe sequence $P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$. This means the system can allocate resources to each process in this specific order without causing a deadlock. Each process completes and releases its resources, enabling the next process to proceed.

Comparison with Theoretical Expectations:

- The result matches the expected output of the classical Banker's Algorithm example
- The Safety Algorithm correctly identified that all processes can finish
- No circular wait exists in this resource allocation state
- The system never needs to enter an unsafe state

Real-World Significance:

- Banker's Algorithm is used in real OS resource managers to prevent deadlock
- It is conservative — it may deny safe requests to stay in a guaranteed safe state
- The trade-off is lower resource utilization in exchange for deadlock-free operation

Output Screenshots

The following screenshots show the execution of the Banker's Algorithm program on Linux (Ubuntu) via VirtualBox.

Input

```
aryan@maurya-VirtualBox: ~/Desktop
aryan@maurya-VirtualBox:~/Desktop$ =====
BANKER'S ALGORITHM - Deadlock Avoidance
=====

Enter number of processes: 5
Enter number of resource types: 3

--- Input Allocation Matrix ---
(Enter 3 values per process, space-separated)

Allocation for P0: 0 1 0
Allocation for P1: 2 0 0
Allocation for P2: 3 0 2
Allocation for P3: 2 1 1
Allocation for P4: 0 0 2

--- Input Maximum Matrix ---
(Enter 3 values per process, space-separated)

Maximum for P0: 7 5 3
Maximum for P1: 3 2 2
Maximum for P2: 9 0 2
Maximum for P3: 2 2 2
Maximum for P4: 4 3 3

--- Input Available Resources ---
Available (3 values): 3 3 2
```

Need Matrix

```
aryan@maurya-VirtualBox: ~/Desktop
=====
KER'S ALGORITHM - Deadlock Avoidance
=====

number of processes: 5
number of resource types: 3

put Allocation Matrix ---
( 3 values per process, space-separated)

Allocation for P0: 0 1 0
Allocation for P1: 2 0 0
Allocation for P2: 3 0 2
Allocation for P3: 2 1 1
Allocation for P4: 0 0 2

put Maximum Matrix ---
( 3 values per process, space-separated)

Maximum for P0: 7 5 3
Maximum for P1: 3 2 2
Maximum for P2: 9 0 2
Maximum for P3: 2 2 2
Maximum for P4: 4 3 3

put Available Resources ---
Available (3 values): 3 3 2

=====
SYSTEM STATE
=====
```

Process	Alloc R0	Alloc R1	Alloc R2	Max R0	Max R1	Max R2	Need R0	Need R1	Need R2
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

Output

```
aryan@maurya-VirtualBox: ~/Desktop

Available Resources: [3, 3, 2]
=====

--- Running Safety Algorithm ---

Process P1 can execute.
Need:      [1, 2, 2]
Work before: [3, 3, 2]
Work after: [5, 3, 2]

Process P3 can execute.
Need:      [0, 1, 1]
Work before: [5, 3, 2]
Work after: [7, 4, 3]

Process P4 can execute.
Need:      [4, 3, 1]
Work before: [7, 4, 3]
Work after: [7, 4, 5]

Process P0 can execute.
Need:      [7, 4, 3]
Work before: [7, 4, 5]
Work after: [7, 5, 5]

Process P2 can execute.
Need:      [8, 0, 0]
Work before: [7, 5, 5]
Work after: [10, 5, 7]

=====
RESULT ANALYSIS
=====

✅ System is in a SAFE STATE.

Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2

Explanation:
The system can allocate resources to each process
in the above order without causing a deadlock.
Each process can complete and release its resources,
allowing the next process to proceed safely.
This matches the theoretical expectation of
Banker's Algorithm – deadlock is avoided by
ensuring the system never enters an unsafe state.

=====
aryan@maurya1-VirtualBox: ~/Desktop$
```


Conclusion

This assignment successfully implemented and analyzed the Banker's Algorithm for Deadlock Avoidance. The program correctly:

- Accepted system input for 5 processes and 3 resource types
- Calculated the Need Matrix using $\text{Need} = \text{Maximum} - \text{Allocation}$
- Implemented the Safety Algorithm to check system safety
- Determined the safe sequence: $P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$
- Confirmed the system is in a SAFE STATE with no deadlock

The implementation demonstrates that the Banker's Algorithm is an effective deadlock avoidance strategy that ensures the system never enters an unsafe state by carefully managing resource allocation decisions.

Code Structure Summary:

- `get_system_input()`: Accepts all user inputs
- `calculate_need()`: Computes $\text{Need} = \text{Maximum} - \text{Allocation}$
- `display_matrices()`: Shows all matrices in tabular format
- `bankers_safety_algorithm()`: Runs the safety check
- `display_result()`: Shows safe sequence and analysis
- `main()`: Controls the overall execution flow